VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Belagavi - 590018



Mini Project Report

Artificial Intelligence and Machine Learning

"Patient Stroke Prediction using Machine Learning"

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by

NAME: PRARTHANA P

USN : 1DT19CS112

Guide:

Mrs. Chethana V

Prof. Dept. of CSE,



Department of Computer Science & Engineering

DAYANADA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

Kanakapura Road, Udayapura, Bangalore-560 082

2022-2023

CONTENTS

Sl. No.	Topic	Page No.
1	**Introduction**	1
	1.1 Introduction	1
	1.2 Motivation	2
2	**System Requirements**	3
	2.1 Hardware Requirements:	3
	2.2 Software Requirements:	3
3	**System Design**	4
	3.1 Implementation	4
	3.2 Preprocessing	4
	3.3 Predictive Modelling	4
	3.4 Algorithm	5
4	**Sample Code**	6
	4.1 Machine Learning Code	6
5	**Conclusion**	10

Chapter 1

Introduction

1.1 Introduction

A stroke occurs when a blood vessel in the brain ruptures and bleeds, or when there's a blockage in the blood supply to the brain. The rupture or blockage prevents blood and oxygen from reaching the brain's tissues.

Without oxygen, brain cells and tissue become damaged and begin to die within minutes.

There are two primary types of strokes:

- Ischemic stroke involves a blockage caused by either a clot or plaque in the artery.
- Hemorrhagic stroke is caused by either a burst or leaking blood .

Symptoms of a stroke show up in the body parts controlled by the damaged areas of the brain vessel That seeps into the brain. The sooner a person having a stroke gets care, the better their outcome is Likely to be.

Stroke symptoms can include:

- Paralysis
- Numbness or weakness in the arm, face, and leg, especially on one side of the body
- Slurred speech
- Confusion, disorientation, or lack of responsiveness
- Vision problems, trouble walking
- Loss of balance or coordination
- Seizures
- Prompt treatment is key to preventing the following outcomes:
- Brain damage
- Long-term disability

- Death

We have witnessed amazing developments in the field of medicine with the aid of technology. With the advent of annotated dataset of medical records, we can now use data mining techniques to Identify trends in the dataset. Such analysis has helped the medical practitioners to make an accurate prognosis of any medical conditions. It has led to an improved healthcare condition and reduced treatment costs.

The use of data mining techniques in medical records have great impact on the fields of healthcare and biomedicine. This assists the medical practitioners to identify the onset of disease at an earlier stage. We are particularly interested in stroke, and to identify the key factors that are associated with its occurrence

Several studies have analyzed the importance of lifestyle types, medical records of patients on the probability of the patients to develop stroke. Machine learning models are also now employed to predict the occurrence of stroke

This is where Machine Learning comes in. We look into various input factors to predict the probability of a patient having a stroke in the future. We try to make the predictions to the highest possible degree of accuracy. These predictions can then be used by the patients to live a more cautious lifestyle, thus reducing the chances of occurrence of a stroke , which could save lives.

1.2 Motivation

Machine learning can efficiently handle making predictions about the chances of occurrence of a stroke, based on various input factors. Information about the patients is collected beforehand and analyzed, using which, it can be used to get useful insights.

They say prevention is better than cure. Even if a disease is not preventable, it is better to catch it early on . This gives patients a better chance of survival. It has been proven that when certain factors when evaluated together, give us high chances of being able to predict whether a patient could have a stoke with certain accuracy. These decision-making models, when provided with the data and information, can deliver excellent error-free predictions, catch important trends in the data and provide actionable insights, which can be used to help take the right precautions. Vast amounts of data can be easily and efficiently analyzed .

CHAPTER 2

System Requirements

2.1 Hardware Requirements:

- Processor : Intel Core i5
- RAM Size : 8 GB
- Keyboard : Standard QWERTY serial
- Mouse : Standard serial or PS/2 mouse
- Hard disk : 500 GB

2.2 Software Requirements:

- Operating System : Windows 10
- Platform : Jupyter Notebook
- Languages Used : Python
- Domain : Machine Learning
- Backend : Python and its various libraries like NumPy, Pandas, Matplotlib, Sklearn.

CHAPTER 3

System Design

3.1 Implementation

The dataset contains 5110 observations with 12 attributes. These 5110 observations are used in pre-processing. We import the dataset into our jupyter notebook.

3.2 Preprocessing

In order to do data analysis, we have recorded data in .csv file by considering id, gender, age, hypertension, heart disease, work type etc. For analyzing data, we eliminate the id column which does not impact in this analysis technique. We check if there is any null values in the dataset using heatmap which is a two-dimensional graphical representation of data where the individual values that are contained in a matrix are represented as colors. The seaborn python package allows the creation of annotated heatmaps which can be tweaked using Matplotlib tools as per the creator's requirement. If null in dataset exists, we need to enter a value to it. We also need to convert the data which is in string format to binary values using get dummies() which is present in pandas library. We perform binary encoding, one hot encoding and KNN Imputation for missing values in the preprocessing step.

3.3 Predictive Modelling

To this preprocessed data, we apply various algorithms like Logistic Regression, K-nearest neighbor. We train it on various algorithms and test the accuracy We have the highest accuracy values for K-Nearest Neighbor algorithm. We handle the class imbalance to ensure we get the right predictions. We do so using oversampling which increases the data. Then we use this data to predict the values this way the training is fair and even for proper prediction without any bia

3.4 Algorithm

K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique. K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems. K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.

It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

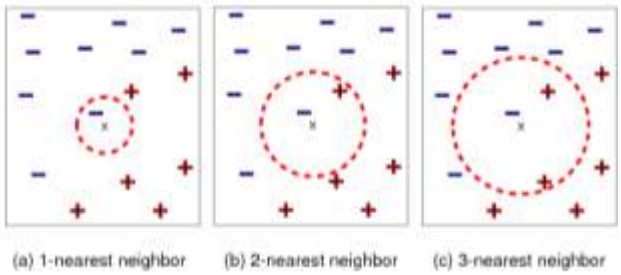


Fig. 1 K-Nearest Neighbors

The following is the pseudo code for KNN:

1. Load the data
2. Choose K value
3. For each data point in the data:
    - Find the Euclidean distance to all training data samples
    - Store the distances on an ordered list and sort it
    - Choose the top K entries from the sorted list
    - Label the test point based on the majority of classes present in the selected points
4. End

CHAPTER 4
Sample Code

4.1 Machine Learning Code

In [43]:
```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

import warnings
warnings.filterwarnings(action='ignore')
```

In [48]:
```
data = pd.read_csv('healthcare-dataset-stroke-data.csv')
print(data.head())
```

```
      id  gender   age  hypertension  heart_disease ever_married  \
0   9046    Male  67.0             0              1          Yes
1  51676  Female  61.0             0              0          Yes
2  31112    Male  80.0             0              1          Yes
3  60182  Female  49.0             0              0          Yes
4   1665  Female  79.0             1              0          Yes

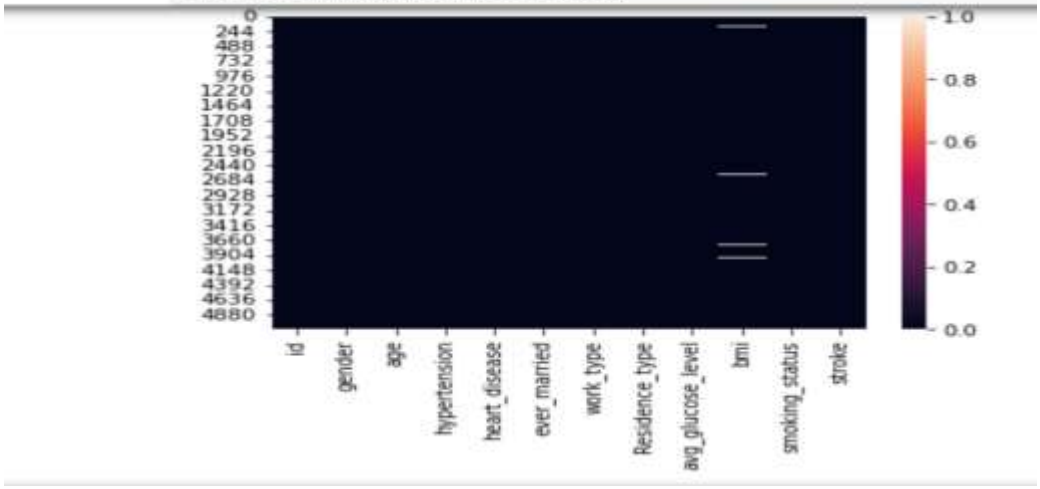
       work_type Residence_type  avg_glucose_level   bmi   smoking_status  \
0        Private          Urban             228.69  36.6  formerly smoked
1  Self-employed          Rural             202.21   NaN     never smoked
2        Private          Rural             105.92  32.5     never smoked
3        Private          Urban             171.23  34.4           smokes
4  Self-employed          Rural             174.12  24.0     never smoked

   stroke
0       1
1       1
2       1
3       1
4       1
```

In [11]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   id                 5110 non-null   int64
 1   gender             5110 non-null   object
 2   age                5110 non-null   float64
 3   hypertension       5110 non-null   int64
 4   heart_disease      5110 non-null   int64
 5   ever_married       5110 non-null   object
 6   work_type          5110 non-null   object
 7   Residence_type     5110 non-null   object
 8   avg_glucose_level  5110 non-null   float64
 9   bmi                4909 non-null   float64
 10  smoking_status     5110 non-null   object
 11  stroke             5110 non-null   int64
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

In [3]:
```
import seaborn as sns
sns.heatmap(data.isnull())
```



In [12]:
```
def preprocessing_inputs(df):
    df=df.copy()

    #drop id col
    df=df.drop('id',axis=1)

    #Binary encoding
    df['ever_married']=df['ever_married'].replace({'No':0 , 'Yes':1 })
    df['Residence_type']=df['Residence_type'].replace({'Rural':0 , 'Urban':1 })

    #One-hot encoding for multi valued string cols
    for column in ['gender','work_type','smoking_status']:
        df=onehot_encode(df,column=column)

    #split into X and y
    y=df['stroke']
    X=df.drop('stroke',axis=1)

    #Train-test split
    X_train,X_test,y_train,y_test = train_test_split(X , y , train_size=0.7, shuffle=True , random_stat

    #KNN imputation of missing values for training dataset only
    imputer=KNNImputer()
    imputer.fit(X_train)  #we do this only for x train because we have seen that y has no missing value
    X_train = pd.DataFrame(imputer.transform(X_train), index=X_train.index, columns=X_train.columns)
    X_test = pd.DataFrame(imputer.transform(X_test), index=X_test.index, columns=X_test.columns)

    #Scale values so they are in the same range ( preffered by some algo)
    scaler= StandardScaler()
    scaler.fit(X_train)
    X_train = pd.DataFrame(scaler.transform(X_train), index=X_train.index, columns=X_train.columns)
    X_test = pd.DataFrame(scaler.transform(X_test), index=X_test.index, columns=X_test.columns)

    return X_train,X_test,y_train,y_test
```

In [13]:
```
def onehot_encode(df,column):   #to change string values
    df=df.copy()
    dummies=pd.get_dummies(df[column],prefix=column)
    df = pd.concat([df,dummies],axis=1)
    df=df.drop(column,axis=1)
    return df
```

```
In [16]: X_train.isna().sum()
         #After running the preprocessing function we have no missing values.
```

```
Out[16]: age                              0
         hypertension                     0
         heart_disease                    0
         ever_married                     0
         Residence_type                   0
         avg_glucose_level                0
         bmi                              0
         gender_Female                    0
         gender_Male                      0
         gender_Other                     0
         work_type_Govt_job               0
         work_type_Never_worked           0
         work_type_Private                0
         work_type_Self-employed          0
         work_type_children               0
         smoking_status_Unknown           0
         smoking_status_formerly smoked   0
         smoking_status_never smoked      0
         smoking_status_smokes            0
         dtype: int64
```

```
In [17]: y_train.value_counts()  #Number of values of 0 and 1. These are the values before we handles the class imbalance
```

```
Out[17]: 0    3411
         1     166
         Name: stroke, dtype: int64
```

```
In [18]: y_train.value_counts()[0] - y_train.value_counts()[1]   #Diff between number of values of 0 and 1
```

```
Out[18]: 3245
```

```
In [20]: oversampled_data = pd.concat([X_train, y_train], axis=1).copy()  #Just adding all the data into one for oversampling process

         num_samples = y_train.value_counts()[0] - y_train.value_counts()[1]
         new_samples = oversampled_data.query("stroke == 1").sample(num_samples, replace=True, random_state=1)

         oversampled_data = pd.concat([oversampled_data, new_samples], axis=0).sample(frac=1.0, random_state=1).reset_index(drop=True)
         #In the end we drop the old indexes so there are no duplicates or wrong info since our sampled data also contains duplicated valu
         #Thus we create or reset new indices for all rows

         y_train_oversampled = oversampled_data['stroke']
         X_train_oversampled = oversampled_data.drop('stroke', axis=1)
```

```
In [21]: oversampled_data.query("stroke==1").sample(num_samples,replace=True)
         #get all the data which has stroke =1 then we use it to create required number of samples, we need to
         #make sure replace is true since we have only 166 values we need around 3500+ values which is a huge number
         #so we keep replacing the values so we have more values that we can use to sample
```

In [21]:
```
oversampled_data.query("stroke==1").sample(num_samples,replace=True)
#get all the data which has stroke =1 then we use it to create required number of samples, we need to
#make sure replace is true since we have only 166 values we need around 3500+ values which is a huge number
#so we keep replacing the values so we have more values that we can use to sample
```

...

In [22]:
```
oversampled_data #since we have alot of duplicate data we need to  delete duplicates
```

...

In [23]:
```
oversampled_data.drop_duplicates()
oversampled_data['stroke'].value_counts() #now we have balanced the classes
```

Out[23]:
```
0    3411
1    3411
Name: stroke, dtype: int64
```

In [17]:
```
models = {
    "                  Logistic Regression": LogisticRegression(),
    "                  K-Nearest Neighbors": KNeighborsClassifier(),
}

for name, model in models.items():   #training our data in these models
    model.fit(X_train_oversampled, y_train_oversampled)
    print(name + " trained.")
```

```
                  Logistic Regression trained.
                  K-Nearest Neighbors trained.
```

In [20]:
```
from sklearn import metrics
from sklearn.metrics import mean_squared_error
print("Model Performance\n-----------------")
for name, model in models.items():
    y_pred = model.predict(X_test)
    print(
        "\n" + name + " Accuracy: {:.3f}%\n\t\t\t\t\t    "\
        .format(accuracy_score(y_test, y_pred) * 100, metrics.precision_score(y_test, y_pred), )
    )
```

```
Model Performance
-----------------

                  Logistic Regression Accuracy: 73.190%


                  K-Nearest Neighbors Accuracy: 85.845%
```

Chapter 5
Conclusion

By the help of AI/ML code using python we can predict any data set given and as per the topic chosen about Patient Stroke Prediction, the above-mentioned code using python helped a lot to predict whether a patient could suffer a stroke, this could save lives. By using K Nearest Neighbor algorithm, we can predict the output to great accuracy.

In the future the accuracy can be increased, and we could develop a system that also predicts the percentage of occurrence of stroke for a patient. More factors could be included for the prediction to ensure the results are through. The overall project would benefit the patients.

They say prevention is better than cure. Even if a disease is not preventable, it is better to catch it early on . This gives patients a better chance of survival. It has been proven that when certain factors when evaluated together, give us high chances of being able to predict whether a could suffer a stroke or not. In general it can help ensures high risk people are taking precautions

Stroke, a cerebrovascular disease, is one of the major causes of death. It causes significant health and financial burdens for both patients and health care systems. The negative impact of stroke in society has led to concerted efforts to improve the diagnosis of stroke. With an increased collaboration of technology and medical diagnosis, caregivers create opportunities for better patient management by systematically gathering various input parameters. Therefore, it is vital to study the interdependency of these risk factors in patients and understand their relative contribution to stroke prediction.

The aim of this project was to apply computational methods using machine learning techniques to predict stroke from various input parameters like gender, age, glucose level, BMI etc. We use various ML models and compare their accuracy.

K-Nearest Neighbours gives us the highest accuracy of 85.845% , which would be the best

.